Geonovum Standaard - Werkversie

MIM-FBM. Metamodel Informatiemodellering voor Fact-Based Modelling

Geonovum Standaard
Werkversie 18 mei 2026

Laatste werkversie:
https://geonovum.github.io/NL-ReSpec-GN-template/

Redacteur:
voornaam achternaam (Geonovum)

Auteur:
voornaam achternaam (Geonovum)

Doe mee:
GitHub Geonovum/NL-ReSpec-GN-template
Dien een melding in
Revisiehistorie
Pull requests

Dit document is ook beschikbaar in dit niet-normatieve formaat: pdf

Samenvatting

TODO: vul in abstract.md een abstract in.

Status van dit document

Dit is een werkversie die op elk moment kan worden gewijzigd, verwijderd of vervangen door andere documenten. Het is geen stabiel document.

Inhoudsopgave

§ 1. Inleiding

§ 2. Toepassingsgebied

§ 3. Normreferenties

§ 4. Termen, definities en afkortingen

§ 4.1 Termen, definities

Term	Definitie
Fiets	Een vervoermiddel met minimaal twee wielen dat met spierkracht via pedalen wordt aangedreven, al dan niet met elektrische trapondersteuning

> NOOT
>
> Geef de term op deze manier aan in de tekst: We gebruiken een fiets om te fietsen.

§ 4.2 Afkortingen

§ 5. Conformiteit

§ 6. Metamodel in Fact-Based Modelling

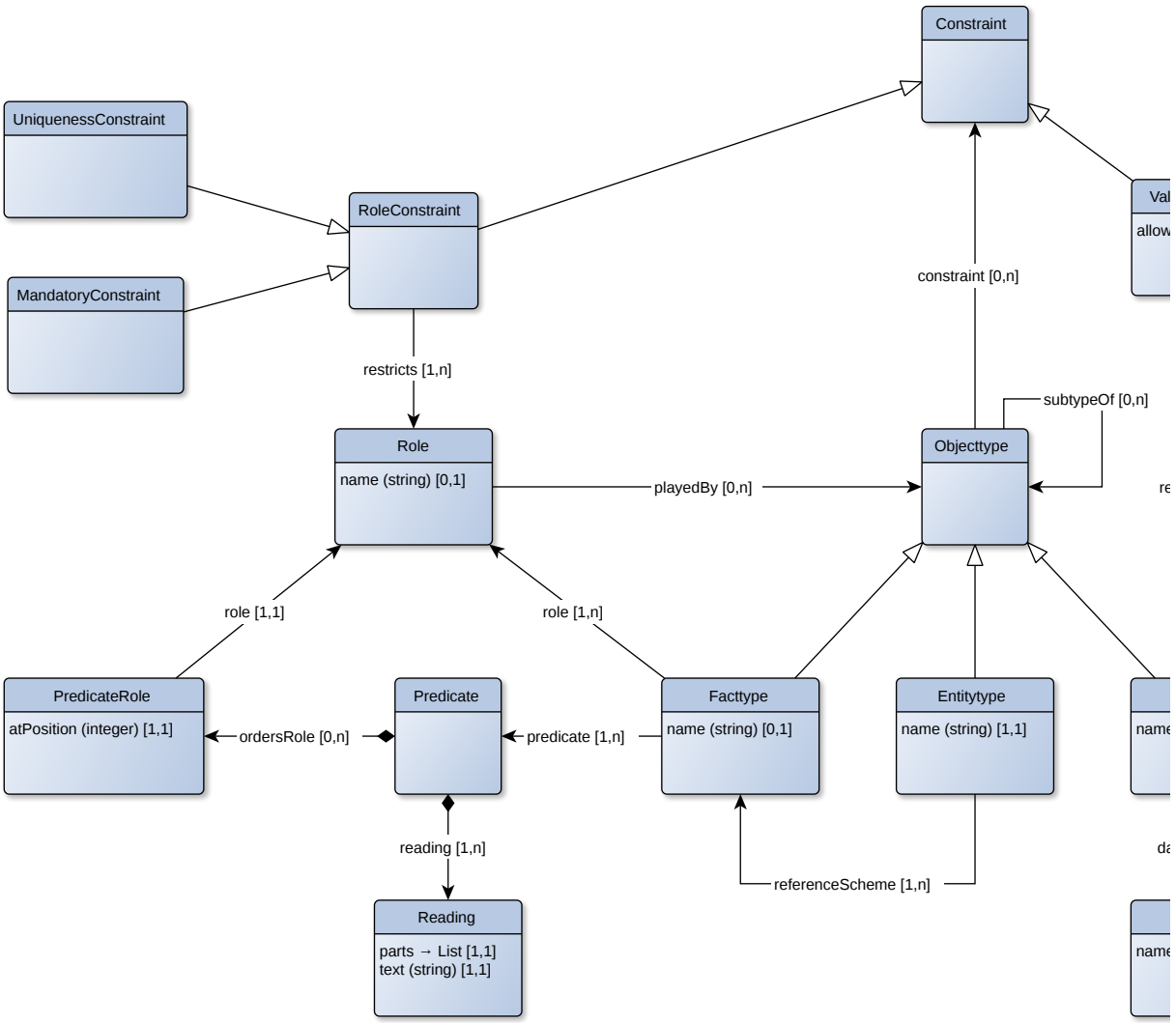
Dit document beschrijft de mapping tussen Fact Based Modeling (FBM) en MIM (Metamodel voor Informatiemodelleren). Op dit moment is het document in het Engelse opgesteld. De reden is het specifieke gebruik van de de Engelstalige termen zoals deze gebruikelijk zijn in Fact Based Modeling. Door het stuk volledig in het Engelse op te stellen, leest het document een stuk prettiger dan het gebruik van Nederlands en Engels door elkaar. In een volgende versie kan wellicht gekozen worden voor een volledig Nederlandse variant.

To perform a mapping between FBM and MIM, it is necessary to have a metamodel (vocabulary) to express a FBM model. From this metamodel, we can describe the mapping to the metamodel of MIM.

Fact Based Modeling has many different dialects, among others: FCO-IM, CogNIAM and ORM. Multiple metamodels have been proposed for FBM, but no actual metamodel has been explicitly defined as ***the*** metamodel for FBM. This document uses a metamodel that is derived from the original notation for FBM as used by the ORM dialect and also used by the FCO-IM dialect.

Not all elements are yet available in this metamodel. We focus on the most fundamental elements.

§ 6.1 Metamodel voor Fact Based Modeling



Objecttypes can be Facttypes, Entitytypes or Valuetypes. Entitytypes refer to classes of real-world objects (Persons, things, places, events). Valuetypes refer to classes of values (like numbers, strings, dates). Facttypes refer to classes of facts about these real-world objects: properties or relations between them.

As Entitytypes refer to specific real-world objects, we need a reference scheme, like you reference a person by his or her full name, or maybe a social security number, or... As such, every Entitytype has one fact type as the preferred reference scheme.

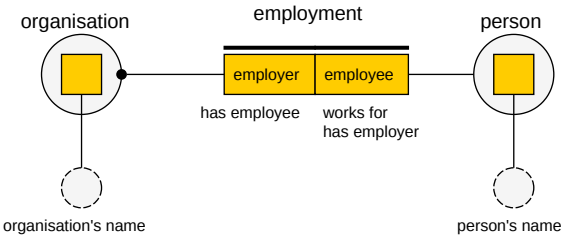
A Valuetype has values that are of a specific datatype, like string or number. These Valuetypes might be constraint by a ValueConstraint. In the current metamodel, only ValueConstraints of the kind "allowed values" are included.

Facttypes can be unary, binary of n-ary. This refers to the roles that are associated to the fact type. Most facttypes are binary. Roles can be played by Entitytypes or Valuetypes. A facttype doesn't have to have a name, but should include at least one Predicate and for that Predicate a PredicateReading.

Predicates are used to state facts from fact types. For each predicate, we can have one or more readings: a sentence in a natural language that states the fact, so it is easy to read by non-modellers.

§ 6.2 Example

Let's look at the example below, a very simple fact based model with two entity types and one facttype:



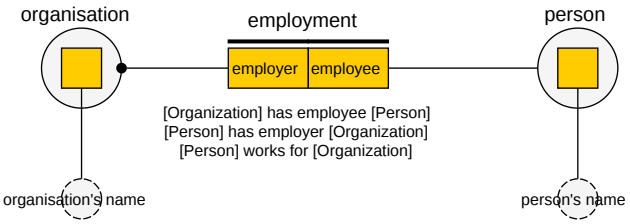
- We introduce an Entitytype with the name "Person" and an Entitytype with the name "Organization"
- We introduce a Facttype with the name "Employment" with two roles
  - One role has name "employee" and is played by the Entitytype "Person"
  - The other role has name "employer" and is played by the Entitytype "Organization"

Let's now introduce the predicates. We will have two predicates:

- The first predicate places the roles in the order {employee, employer}, this predicate has two readings:
  - The reading ".. works for .." (abbreviated to "works for" as in binary fact types an infix predicate reading is expected)
  - The reading ".. has employee .." (abbreviated to "has employee")
- The second predicate places the roles in the order {employer, employee}, this predicate has one reading:
  - The reading ".. has employee .." (abbreviated to "has employee")

Predicates are used to verbalise the knowledge specified by the facttype. At least one predicate should be present. Names of fact types or roles are not actually needed, but can be useful to denote specific terms used in the domain.

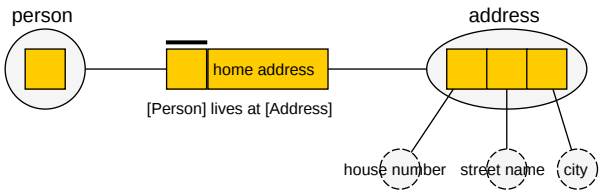
Predicate readings can be abbreviated, but the full reading is also allowed. This means that the notation below depicts exactly the same facttype:



Uniqueness constraints are role constraints that are used to specify which combinations of roles will create a unique fact or refer to a unique entity. A uniqueness constraint is depicted as a line above the role. Mandatory constraints are used to specify that every single fact or entity should play that particular role. For that reason, mandatory constraints are also known as totality constraints. Mandatory constraints are depicted as a dot at the end of the line that depicts which objecttype plays the role.

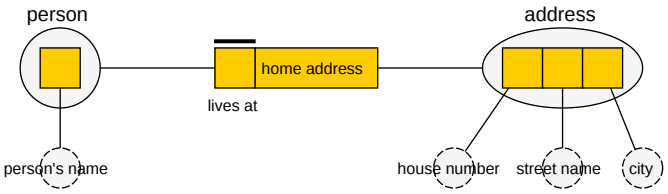
Facttypes can be objectified. As such, a facttype can also play roles in other fact types. An objectified facttype always has a name. In the example above the entity type "Person" and "Organization" are actually drawn as objectified facttypes: the only difference is semantically: entity types refer to real-world things, whereas facttypes (only) refer to facts.

At least one predicate reading should exists for any facttype, but role names and facttype names are optional, as is made clear by the following example:



It doesn't make sense to talk about the name of the facttype ("the fact that a person lives somewhere" isn't a named fact!) and although you might speak about a "resident" as the role that is played by a person, this isn't a *proper* property of an address. We could even leave out the name of the role that is played by an address ("home address"), as the predicate reading is en-

ough. And even if we don't want the predicate reading "lives at", we can always create a predicate reading using the role name: "[Person] has home address [Address]". When we use the shorthand notation for the predicate reading, it looks very elegant:



6.3 Mapping of FBM to MIM

In the mapping below, we consider an fbm:Objecttype as an objectified fbm:Facttype or a fbm:Entitytype with its reference scheme, as the mapping of those two are identical in the current version of MIM, so:

- An objectified facttype can *have* roles and can *play* roles;
- An entitytype can *have* roles via its reference scheme and can *play* roles.

Valuetypes can only *play* roles and can never *have* roles. Only valuetypes are mapped to a datatypes.

6.3.1 fbm:Objecttype to mim:Objecttype

- All fbm:Objecttype are translated to mim:Objecttype.
- The fbm:name of the fbm:Objecttype will be used as mim:name.

6.3.2 fbm:subtype to mim:Generalisatie

- All fbm:subtypeOf are translated to mim:Generalisatie.
- The mim:subtype will map to the subject of the fbm:subtypeOf.
- THe mim:supertype will map to the object of the fbm:subtypeOf.

Mark that fbm:subtype can be used for both fbm:Objecttypes as fbm:Valuetypes. This corresponds with MIM, as mim:Generalisatie can be used for mim:Objecttype and mim:Datatype.

6.3.3 fmb:Objecttype with fbm:role played by fbm:Valuetype to mim:Attribuutsoort

- All fbm:role that are played by fbm:Valuetype are translated to mim:Attribuutsoort.
- This mim:Attribuutsoort will be the mim:attribuut of the fbm:Objecttype that contains this fbm:Role.
- The fbm:name of the fbm:Role will be used as mim:name.
- If the name doesn't exists, the name of the fbm:Valuetype is used

6.3.4 fbm:Objecttype with fbm:role played by fbm:Objecttype to mim:Relatiesoort

- All fbm:role that are played by fbm:Objecttype are translated to mim:Relatiesoort.
- This mim:Relatiesoort will have as mim:bron the fbm:Objecttype that contains this fbm:Role.
- This mim:Relatiesoort will have as mim:doel the fbm:Objecttype that plays this fbm:Role.
- The fbm:name of the fbm:Role will be used as mim:name.

- If this name doesn't exists, The textual predicate reading part will be used, for the predicate in which the mim:doel objecttype is in the second position.

§ 6.3.5 Unary fbm:Facttype

- All unary fbm:Facttypes are translated to mim:Attribuutsoort.
- The mim:type of this mim:Attribuutsoort will be mim:Boolean.
- The fbm:name of the fbm:Role will be used as mim:name.
- If this name doesn't exists, the textual predicate reading part will be used.

§ 6.3.6 Binary fbm:Facttype in which one of the fbm:roles is played by a fbm:Valuetype to mim:Attribuutsoort

- All binary fbm:Facttypes in which one of the fbm:roles is played by a fbm:Valuetype are translated to mim:Attribuutsoort.
- This mim:Attribuutsoort will be the mim:attribuut of the fbm:Objecttype that plays the other fbm:role.
- The fbm:name of the fbm:Role will be used as mim:name.
- If this name doesn't exists, the textual predicate reading part will be used, for the predicate in which the valuetype is in the second position.

§ 6.3.7 Binary fbm:Facttype in which both fbm:roles are played by fbm:Objecttypes to mim:Relatiesoort

- All binary fbm:Facttypes in which both fbm:roles are played by fbm:Objecttypes are translated to mim:Relatiesoort.
- The fbm:name of the fbm:Facttype will be used as mim:name.
- This mim:Relatiesoort will have as mim:bron the fbm:Objecttype that playes the first role in the predicate
- This mim:Relatiesoort will have as mim:doel the other objecttype that playes the second role in the predicate
- If more than one predicate exists, duplicate mim:Relatiesoorten will be created, in that case the predicate reading text will be used as mim:name
- If a fbm:Role has an fbm:name, that name is used for the corresponding mim:Relatierol.

§ 6.3.8 n-Ary fbm:Facttype

All n-Ary fbm:Facttypes are considered objectified facttypes and treated as such (e.g.: as fbm:Objecttypes)

MIM doesn't have solution for n-Ary relationships, so we must treat theses as objectified facttypes. In some cases, an n-Ary fbm:Facttype could be translated using a mim:Relatieklasse, but this is currently beyond the scope of this document.

§ 7. Niet-normatieve deel

Dit onderdeel is niet-normatief.

Bijvoorbeeld een introductie is niet normatief.

> NOOT: index
>
> Dit hoofdstuk is toegevoegd met `class="informative"` in `config.js`.

§ 7.1 Een subparagraaf

Sleutel	Waarde
key	value

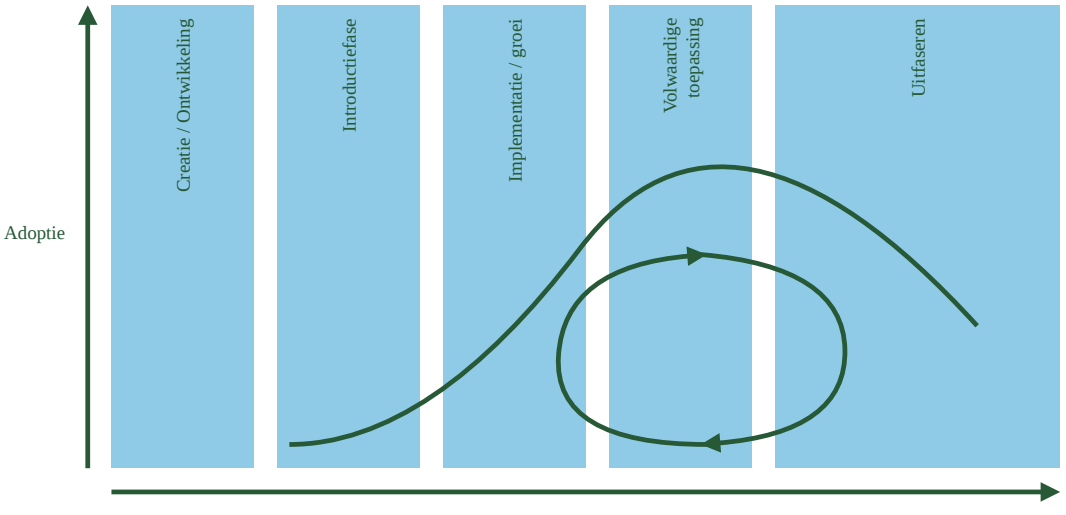
§ 8. Meer inhoud

§ 8.1 Definities

Definitie: Een definitie is een beschrijving van een woord. Een ander woord voor *definitie* is betekenis of beschrijving.

§ 8.2 Afbeeldingen

Afbeeldingen krijgen een nummer en vermelding in de figurenlijst 11. Lijst met figuren.



Figuur 1 Onderschrift

§ 8.3 Referenties

Referenties kunnen op drie plaatsen staan:

- In SpecRef Specref
- In de organisatie configuraties [*SemVer*]. Deze zijn te vinden op tools.geostandaarden.nl.
- In het document, zoals [*MIM12*]. Deze zijn te vinden in `js/config.js`.

Referentie uit organisatie lijst [*SemVer*] of de locale lijst [*MIM12*]. Lijst staat in `organisation-config.js`. Alleen referenties die in de tekst voorkomen worden getoond.

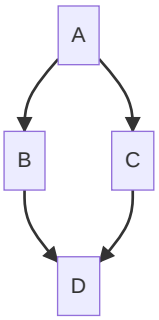
We gebruiken een definitie om een woord te omschrijven.

§ 8.4 Optioneel

De onderstaande secties (*Conformiteit* e.d.) zijn optioneel, zie `index.html`:

```
<body>
  <section id="abstract" data-include-format="markdown" data-include="abstract.md"></sectior
  <section id="sotd"></section><!-- Wordt automatisch gevuld -->
  <section data-include-format="markdown" class="informative" data-include="ch01.md"></secti
  <section data-include-format="markdown" data-include="ch02.md"></section>
  <!-- Hieronder optionele secties. Worden automatisch gevuld -->
  <section id='conformance'></section>
  <section id='tof'></section>
  <section id="index"></section>
</body>
```

§ 9. Mermaid diagram



Figuur 2 Mermaid voorbeeld

§ 10. Conformiteit

Naast onderdelen die als niet-normatief gemarkeerd zijn, zijn ook alle diagrammen, voorbeelden, en noten in dit document niet-normatief. Verder is alles in dit document normatief.

§ 11. Lijst met figuren

§ A. Index

§ A.1 Termen gedefinieerd door deze specificatie

§ A.2 Termen gedefinieerd door verwijzing

§ B. Referenties

§ B.1 Normatieve referenties

[MIM12]
MIM - Metamodel Informatie Modellering (Versie 1.2). Geonovum. 2024-06-13. Definitief. URL: https://docs.geostandaarden.nl/mim/def-st-mim-20240613/

[SemVer]
Semantic Versioning 2.0.0. T. Preston-Werner. June 2013. URL: https://semver.org

↥